GitHub Issues: Overview and Project Integration

1. What is GitHub Issues?

GitHub Issues is a built-in tracking feature available in every GitHub repository, used to record and manage tasks, bugs, and any work item that needs attention. It is not a separate tool that has to be installed — it exists automatically as a tab (alongside Code, Pull Requests, Actions, and Settings) the moment a repository is created.

2. Purpose

GitHub Issues exists to answer one central question at all times: **"What still needs to be fixed or done, and who is responsible for it?"**

Without a system like this, work items tend to live in places that don't persist or aren't visible to the whole team — a comment made in a meeting, a message in a group chat, or something remembered only by one person. GitHub Issues solves this by giving every problem a permanent, visible, trackable home that the whole team can see at once.

What it guarantees

Guarantee	Explanation
Persistence	An issue does not disappear once seen — it stays open until resolved, and remains as a historical record even after closing
Accountability	Every issue has a named assignee, making responsibility explicit rather than assumed
Shared visibility	The entire team sees the same up-to-date list, instead of everyone holding a different partial picture

Notification vs. Issue

A common point of confusion is treating an issue like a notification. They are fundamentally different:

	Notification	GitHub Issue
Lifespan	Momentary — appears once, then gone	Permanent — stays until closed, and remains as history afterward
Purpose	Alerts that something happened	Tracks a problem until it is actually resolved
Has a status?	No	Yes (Open / Closed)
Has an owner?	No	Yes (assignee)
Discussion thread?	No	Yes (comments)
Can be ignored/lost?	Easily	No — stays visible until explicitly closed

3. Core Components of an Issue

Field	Purpose
Title	One-line summary of the problem
Body	Full description; supports Markdown (headers, code blocks, checklists, images)
Number	Auto-assigned, permanent ID (#1, #2, #3…) — never reused
Assignee	The person or team responsible for resolving it
Labels	Tags for categorization/filtering (e.g., `bug`, `critical`, `security`)
Milestone	Groups issues under a larger goal or deadline
State	Open or Closed
Comments	A permanent discussion thread attached to that specific issue

4. Lifecycle / Workflow

```
1. CREATE  → someone (or automation) opens a new issue
2. DISCUSS → team comments, clarifies, investigates
3. ASSIGN  → a person/team is tagged as responsible
4. WORK    → the assignee resolves the underlying problem
5. LINK    → the fix references the issue (e.g., "fixes #14" in a commit)
6. CLOSE   → issue is closed automatically or manually, and remains as a record
```

5. Key Features

Markdown support — issues can contain formatted text, code blocks, checklists, and images
Cross-linking — issues can reference other issues, pull requests, and commits, building a traceable history
Labels and filters — allow teams to instantly filter large issue lists (e.g., show only `critical` and `open`)
REST API — issues can be created, updated, and closed programmatically, not only through the web interface

6. Why It Matters

Nothing gets lost — every reported problem becomes a permanent record
Ownership is explicit — the assignee field removes ambiguity about who is responsible
Priority is visible — labels let anyone filter to what's urgent instantly
Traceability — a fix can always be linked back to the original problem
Free and requires zero setup — included automatically with every GitHub repository

7. Role in This Project (Risk Prioritization and Deduplication Pipeline)

GitHub Issues is the **final stage** of the vulnerability risk pipeline, sitting after scanning, normalization, deduplication, threat-intel enrichment, and risk scoring:

```
Raw scanner output → Normalize → Deduplicate & filter → Enrich with threat intel
  → Score & rank (0-100) → Auto-generate GitHub Issues + dashboard
```

Every stage before this produces data (a ranked list of risky findings in JSON). GitHub Issues is where that data becomes an actionable, owned task that a developer will actually see and act on.

How it is integrated

A script (`generate_tickets.py`) reads the scored findings output and, for every finding above a defined risk-score threshold, calls the GitHub REST API to automatically create an issue containing:
Title — the vulnerability name and risk tier
Body — the full description, affected asset, and the score breakdown (CVSS, EPSS, KEV status) so the ranking is explainable, not a black box
Assignee — looked up from a predefined asset-to-owner mapping
Labels — the risk tier and an SLA indicator (e.g., `critical`, `sla:7-days`)
The script also checks existing open issues before creating new ones, so re-running the pipeline does not generate duplicate tickets for findings that already have one.

Why GitHub Issues specifically (and not Jira or a custom dashboard)

Free and zero setup — the project's code already lives on GitHub, so Issues is available automatically with no separate account, hosting, or licensing cost.
Simple, well-documented, free API — programmatic ticket creation requires only a few lines of code, which fits an automated pipeline.
Directly satisfies the project requirement — "ticket-ready output with owners/SLAs" maps exactly onto Issues' native assignee and label/milestone features.
Meets developers where they already work — findings become visible in the exact tool developers check daily, rather than a separate report that can be forgotten.